Lab #1
Classes, Objects, Encapsulation

Module: Computer Programming

Preparation

- Create a new folder for this lab called `lab1`.
- There are **5 questions**, worth **0.5 point each**.
- Besides making the code work, you should also **minimize code duplication**.

Quick Reference: Java's `String` Data Type

- `String` is a non-primitive (reference) data type.
- Declare a String variable:

```
String name;
```

String initialization:

```
String name = new String("Oak");
String name = "Oak";
```

Copying one String value to another String variable:

```
String name1 = "Oak";
String name2 = name1;   // now name2 = "Oak"
```

Testing whether two Strings contain the same content:

```
String a, b;
a = "Oak";
b = "Oak";

// DON'T:
a == b;                 // Why? We will see in next week's lecture.

// DO:
a.equals(b);
a.equals("Oak");
```

Q1: Warm Up — 0.5 point

Write a Java class `Book` with the following features.

Attributes

- `title`: the title of the book, of type `String`.
- `author`: the author's name, of type `String`.
- `price`: the book price, of type `double`.

Constructor

- `public Book(String title, String name, double price)`: create a `Book` object by setting the attributes to the passed values.

Methods

- `public void setTitle(String title)`: set the title of the book.
- `public void setAuthor(String author)`: set the name of the book's author.
- `public void setPrice(double price)`: set the price of the book.
- `public String getTitle()`: return the title of the book.
- `public String getAuthor()`: return the author's name.

Encapsulation requirement: The `Book` class must be fully encapsulated.

Write a separate class `BookDemo` with a `main()` method that:

1. Creates a `Book` titled **"Developing Java Software"**, with author **Russel Winder** and price **79.75**.
2. Prints the book's title and author to standard output using `System.out.println`.

Q2: Employee — 0.5 point

Create a class called `Employee` with the following requirements:

1. It has three attributes:
   - first name (type `String`),
   - last name (type `String`), and
   - monthly salary (type `double`).
2. It has two constructors:
   - one with three input arguments that initializes all three attributes; and
   - one with two input arguments that initializes the first and last names while setting the monthly salary to the default value `0`.

3. Use `this` to make your constructor implementation shorter.

4. The class must be fully encapsulated.

5. Each attribute should be readable and modifiable from code outside the class.

6. The monthly salary may be updated **only if the new value is positive**. If the new value is negative or zero, do not update the salary.

Then write a test application named `EmployeeTest` that tests `Employee`'s capabilities. The test should include:

1. Creating one `Employee` object and reading and modifying all of its attributes while respecting the conditions above.

2. Creating two `Employee` objects and displaying each object's yearly salary. You may need to add a new method to `Employee`.

3. Using the same two objects, giving each employee a 10% raise and displaying each employee's yearly salary again. You may need to add a new method to `Employee`.

Q3: Employee Team — 0.5 point

Working on top of `Employee` from Q2, implement a Java class `EmployeeTeam` with the following requirements.

Attributes

- `boss` (type `Employee`)
- `employee` (type `Employee`)

Constructor

- One constructor with two input arguments of type `(Employee, Employee)`, used to initialize `boss` and `employee`.

Methods

1. `printEmployeeDetails()`: print the first name, last name, and salary of `employee`.

2. `printAllEmployeesDetails()`: print the first name, last name, and salary of both `boss` and `employee`.

3. `updateSalaryOfEmployee(String firstname, double newSalary)`: update the monthly salary of the `Employee` object(s) (i.e., `boss` and/or `employee`) whose first name is `firstname`. Only update the salary if `newSalary > 0`.

4. `giveRaiseToAllEmployees()`: give a 10% raise to both `boss` and `employee`.

Write a class called `EmployeeTeamTest` to test `EmployeeTeam`'s features with respect to the requirements above.

Q4: Account — 0.5 point

You are given the following partial implementation of the `Account` class. `Account` represents a banking account containing an account name and a balance.

```
public class Account
{
    private String name;
    private double balance;

    public Account(String name, double balance)
    {

    }

    public void deposit(double depositAmount)
    {

    }
}
```

Finish the implementation according to the following requirements:

1. You may **not modify the existing code**, but you may add new code. Implementing the constructor or `deposit` method is allowed.
2. The class must be fully encapsulated.
3. An `Account` object cannot be initialized with a non-positive balance. If an attempt is made to initialize an account with a non-positive balance, set `balance` to `0`.
4. The `name` attribute should be readable and modifiable from outside the `Account` class.
5. The `balance` attribute should be readable from outside the class, but not directly modifiable except through `deposit`.
6. `deposit(double depositAmount)` should add `depositAmount` to the current balance **only if** `depositAmount > 0`. If the input is invalid, the balance remains unchanged.

Write a test application called `AccountTest` to test `Account`'s functionality with respect to the requirements above.

Q5: Account List — 0.5 point

Create a class called `AccountList` with the following requirements.

Attribute

- One attribute called `accounts`, of type `Account[]` (see Q4).

Constructor

- Takes one integer input argument specifying the maximum size that `accounts` can have.

Methods

1. `appendAccount`: takes one input argument of type `Account`.
   - If `accounts` is not full (i.e., appending the new account does not exceed the maximum size specified in the constructor), append the input account and return `true`.
   - If `accounts` is already full, do nothing and return `false`.
2. `getAccount`: takes one integer input argument `idx`, representing an array index.
   - Return `accounts[idx]` if there are at least `idx + 1` appended elements.
   - Otherwise, print `"Input index exceeds the number of appended elements"` and return `null`.

Q5 Test Example

```
public class AccountListTest {
    public static void main(String[] args) {
        AccountList al = new AccountList(3);

        // success should be true
        boolean success = al.appendAccount(new Account("Oak", 1.0));
        success = al.appendAccount(new Account("Two", 0.1));

        // It should print:
        // "Input index exceeds the number of appended elements"
        Account account = al.getAccount(2);
        System.out.println(account); // should print null

        account = al.getAccount(1);
        // Or whatever getter is specified. It should print 0.1.
        System.out.println(account.getBalance());

        success = al.appendAccount(new Account("tmp", 0));
        success = al.appendAccount(new Account("tmp2", 0));
        System.out.println(success); // it should print false
    }
}
```

Submission

1. Take screenshots of the output of all questions.
2. Make sure all files (Java programs and output screenshots) are placed under the `lab1` folder.
3. Push all files to GitHub.
4. Submit your GitHub link under **"Lab 1: Classes, Objects, Encapsulation"** in LMS.

End of Lab #1